

CSCE 671 HUMAN-COMPUTER INTERACTION · MODULE 05 · LECTURE NOTES

Interaction Design Principles

The rules, and what each one costs.

MODULE OUTCOMES

- State the principles and the mechanism behind each.
- Explain where they conflict and how to decide.
- Explain visibility, feedback, and constraints concretely.
- Explain defaults and why they dominate.
- Apply the heuristics as a review method.

1 The principles

Principle	The mechanism it addresses	Where that is established
Visibility	Closes the gulf of execution — the user can see what is possible.	Module 01 §1
Feedback	Closes the gulf of evaluation , and must overcome change blindness.	Module 01 §1, Module 02 §4
Consistency	Preserves learned conventions and reduces the memory load.	Module 01 §3, Module 03 §2
Constraints	Makes the error impossible — the strongest of the three error fixes.	Module 01 §4
Recognition over recall	Exploits the memory asymmetry.	Module 03 §3
Reversibility	Makes the error recoverable, requiring nothing at the moment of the error.	Module 01 §4
Good defaults	Requires nothing of the user at all.	Module 05 §3

A PRINCIPLE WITHOUT A MECHANISM IS A PREFERENCE, AND A PRINCIPLE WITH ONE IS A PREDICTION

'Be consistent' as a bare injunction can be argued with indefinitely, and frequently is. **'Inconsistency invalidates learned conventions and therefore raises the error rate' is a claim you can test** (Module 03 §4's predictable rates).

So stating the mechanism converts a design argument into an empirical question — which is the only way such arguments actually get settled (Module 01 §2's designer blindness: neither party is a reliable witness).

And it tells you when the principle does not apply. Consistency with a bad convention preserves the wrong learning — so the mechanism tells you the exception, where a rule would not.

Which is why this course states mechanisms rather than rules — a rule you know the reason for is one you can apply to a case nobody anticipated, and interface design is mostly cases nobody anticipated.

2 Conflicts

- **Visibility against simplicity. Showing everything makes nothing visible** (Module 02 §3's emphasis count) — so **visibility means visibility of *what matters now***, which requires knowing what that is.
- **Consistency against local optimality. The locally better treatment breaks the established convention, and the convention is usually worth more** (Module 01 §3's invalidated learning) — but not always, and that is the judgement.
- **Constraints against flexibility. Preventing the error also prevents the legitimate unusual case** — which is the tension inside every validation rule, and is why over-strict validation produces workarounds (CSCE 701 Module 11 §4).
- **Efficiency for experts against learnability for novices** (Module 03 §3) — **resolved by providing both paths rather than by compromising between them**, which is the one conflict in this list with a clean answer.
- **And none of the others resolves in general — which is why Module 07's observation is the arbiter** rather than a stronger principle. **The conflicts are the interesting part of this material**: anybody can apply a principle in isolation, and **design judgement is knowing which one yields in a particular case**.

3 Defaults

THE DEFAULT IS THE SETTING ALMOST EVERYONE HAS, SO IT IS THE DESIGN

A small minority of users **change any given setting**, across essentially every system ever measured — so the default is, in practice, the **configuration of your system as it is actually used**.

Which makes it the **only principle in §1's table that requires nothing whatsoever of the user** — no attention, no memory, no learning, no reading, and no decision.

And it is why **CSCE 701 Module 11 §3 and CSCE 713 Module 10 §2 both arrive at exactly the same conclusion from a security direction: the default is the intervention that works at population scale**, and three courses converging on it is worth noticing.

So **choose defaults for the common case and for the safe case** — and **when those two conflict, say which you chose**, because an **unsafe but convenient default is a decision about other people's risk** taken on their behalf (Module 12).

- **For the most common case**, which requires actually knowing what that is — and **guessing is where defaults usually go wrong**, because the designer's own usage is not representative (Module 01 §2).
- **For the safe case wherever the stakes are asymmetric — a destructive default is wrong even if it is what most people want most of the time**, because the cost of the minority case is not comparable.
- **Pre-fill rather than leave blank**, wherever the system already knows or can guess the answer — which is **Module 03 §2's show-it rule** applied to input rather than to display.
- **And make the default visible and changeable**, because **a hidden default is a decision the user cannot find** and therefore cannot disagree with — which is a small-scale version of Module 12's concern.
- **Then measure how often it is changed**, which tells you directly whether you chose correctly. **Measuring how often a default is changed is the cheapest available evidence that you chose wrong — and almost nobody instruments it**, which leaves a free signal unused.

4 Heuristic review

HEURISTIC EVALUATION FINDS MANY PROBLEMS CHEAPLY AND MISSES THE ONES THAT MATTER MOST

Several reviewers independently walk the interface against a list of principles and record the violations they find — which takes hours rather than days and requires no participants at all, making it the cheapest method in the course.

And it finds a great deal: inconsistencies, missing feedback, poor error messages, violated platform conventions, and unlabelled controls — all of which are real problems worth fixing and all of which a usability test would also have found more expensively.

But it does not find what actual users will do. A reviewer knows what the interface is *for* (Module 01 §2 again), so they cannot discover that the task model itself is wrong — that the user wants to do something the design never contemplated — which is the expensive class of problem.

So: use it before testing, to remove the obvious problems so that the test finds the interesting ones — and never instead of testing, which is the substitution it invites because it is so much cheaper and produces such a satisfying list.

- Use three to five reviewers independently, and then merge their lists — a single reviewer finds only a minority of the problems, and the overlap between reviewers is surprisingly low, which is the empirical finding that makes multiple reviewers necessary rather than merely better.
- Walk specific tasks rather than reviewing the interface as a catalogue — which keeps the review grounded in what somebody would actually be trying to accomplish and avoids a list of decontextualised nitpicks.
- Record the violated principle and its mechanism for each finding (§1), so that each one is actionable and so that a disagreement about it is a disagreement about a mechanism rather than about taste.
- And rate severity as frequency times impact, so that the list can be prioritised rather than merely produced — an unprioritised list of forty findings gets ignored entire.
- Then fix the obvious ones and test (Module 07). The low overlap between reviewers is the finding that makes three to five necessary — one reviewer's list is not a subset of the problems, it is a fairly arbitrary sample of them.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Nielsen — Ten Usability Heuristics (free)	§1's list in its canonical form, and the one most reviews are run against.
Norman — The Design of Everyday Things, chapters 4 and 7	§1's constraints and §2's conflicts, with the mechanisms developed properly.

Resource	What it covers for this module
Nielsen & Molich — Heuristic evaluation of user interfaces (free)	§4 in the original — including the measured overlap between reviewers, which is the key number.
Thaler & Sunstein, and the default-effect literature	§3's effect outside software — where the default-dominance finding was established.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **For each of the seven principles**, state the mechanism without looking.
- 2 **Find a case where consistency is wrong**, and justify breaking it from the mechanism.
- 3 **Find each of the four conflicts** in an interface you use.
- 4 **Resolve one of them** and say what you gave up.
- 5 **List every default in your own software** and say whether it is the common case or the safe one.
- 6 **Find a destructive default** somewhere and propose the change.
- 7 **Instrument one default** and measure the change rate.
- 8 **Run a heuristic review** of an interface, alone.
- 9 **Have two other people do it independently** and measure the overlap.
- 10 **Rate the merged findings by frequency times impact.**

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name seven principles and the mechanism behind each.
- 2 Why does stating the mechanism matter?
- 3 How does a mechanism tell you the exception?
- 4 Name four conflicts and which has a clean resolution.
- 5 Why is the default the design?
- 6 Why do three courses converge on defaults?
- 7 Give five rules for choosing defaults.
- 8 What is the cheapest evidence you chose a default wrong?
- 9 What does heuristic review find, and what can it not find?

10 Why are three to five reviewers necessary?